Data Structures Unit 3 - Stacks and Queues

Long Answer Question Bank

Q1: Explain Stack as an Abstract Data Type (ADT) with its operations, sequential organization using arrays, and implementation algorithms.

Answer:

1. Stack as Abstract Data Type (ADT)

A **stack** is a restricted or controlled linear data structure where insertion and deletion operations are performed at only one end called the **top of the stack**. It follows the **Last In First Out (LIFO)** principle, meaning the last element inserted is the first one to be removed.

Key Characteristics:

- Homogeneous collection of elements
- Only the top element is accessible
- Insertion and deletion occur at the same end (top)
- Real-world analogy: Stack of trays in a cafeteria, pile of books, browser history

Position in Data Structure Hierarchy:

```
Data Structures
    ├── Linear Data Structures
    │   ├── Arrays
    │   ├── Linked Lists
    │   ├── Stacks (Restricted Linear Structure)
    │   └── Queues (Restricted Linear Structure)
    └── Non-Linear Data Structures
```

2. Stack ADT Specification

```
structure STACK (item)
   declare CREATE() → stack
           ADD(item, stack) → stack
           DELETE(stack) → stack
           TOP(stack) → item
           ISEMPS(stack) → boolean;

   for all S ∈ stack, i ∈ item let
      ISEMPS(CREATE) ::= true
```

```
        ISEMPS(ADD(i,S)) ::= false
        DELETE(CREATE) ::= error
        DELETE(ADD(i,S)) ::= S
        TOP(CREATE) ::= error
        TOP(ADD(i,S)) ::= i
end
end STACK
```

Operations Defined:

- **CREATE()**: Initializes an empty stack
- **ADD(item, stack)**: Adds element to top (also called PUSH)
- **DELETE(stack)**: Removes top element (also called POP)
- **TOP(stack)**: Returns top element without removing it
- **ISEMPS(stack)**: Checks if stack is empty

3. Sequential Organization Using Arrays

Implementation Strategy: Stack can be represented using a one-dimensional array with a variable `top` that points to the index of the topmost element.

Structure Definition:

```c
#define MAX_SIZE 100

typedef struct {
    int items[MAX_SIZE];
    int top;
} Stack;
```

Memory Representation:

```
Index:  0   1   2   3   4   5   6
      [78] [56] [34] [ ] [ ] [ ] [ ]
                ↑
               top
```

Key Properties:

- **Bottom**: Index 0 (fixed position)
- **Top**: Variable index indicating current top element
- **Initial State**: top = -1 (empty stack)

- **Full Condition**: top == MAX_SIZE - 1
- **Empty Condition**: top == -1

4. Basic Stack Operations - Detailed Implementation

4.1 isEmpty() Operation

Purpose: Check if stack has no elements

Algorithm:

```
Algorithm isEmpty()
{
   if (top == -1)
      return true;
   else
      return false;
}
```

Time Complexity: O(1)
Space Complexity: O(1)

4.2 isFull() Operation

Purpose: Check if stack has reached maximum capacity

Stack Overflow: Condition resulting from attempting to push an element onto a full stack.

Algorithm:

```
Algorithm isFull()
{
   if (top == MAXSIZE - 1)
      return true;
   else
      return false;
}
```

Time Complexity: O(1)
Space Complexity: O(1)

4.3 Push Operation

Purpose: Add a new element to the top of the stack

Preconditions:

- Stack has been initialized
- Stack is not full

Postconditions:

- newItem is at the top of the stack
- top is incremented

Algorithm:

```
Algorithm push(item)
{
   if (!isFull())
   {
      top = top + 1;
      stack[top] = item;
   }
   else
      print "Stack Overflow";
}
```

C Implementation:

```c
void push(int item) {
   if (top >= MAX_SIZE - 1) {
      printf("Stack Overflow\n");
      return;
   }
   stack[++top] = item;
}
```

Time Complexity: O(1)
Space Complexity: O(1)

4.4 Pop Operation

Purpose: Remove and return the top element from stack

Stack Underflow: Condition resulting from attempting to pop an empty stack.

Preconditions:

- Stack has been initialized
- Stack is not empty

Postconditions:

- Top element has been removed
- top is decremented

Algorithm:

```
Algorithm pop()
{
   if (!isEmpty())
   {
      temp = stack[top];
      top = top - 1;
      return temp;
   }
   else
      print "Stack Underflow";
}
```

C Implementation:

```c
int pop() {
   if (top < 0) {
      printf("Stack Underflow\n");
      return -1;
   }
   return stack[top--];
}
```

Time Complexity: O(1)
Space Complexity: O(1)

---

5. Step-by-Step Stack Operation Examples

Initial State: Empty stack (top = -1)

```
Operation: Push(50)
Stack: [50] [ ] [ ] [ ] [ ] [ ] [ ]
       top=0
Empty Locations: 6
```

```
Operation: Push(25)
Stack: [50] [25] [ ] [ ] [ ] [ ] [ ]
           top=1
Empty Locations: 5
```

```
Operation: Push(78)
Stack: [50] [25] [78] [ ] [ ] [ ] [ ]
                top=2
Empty Locations: 4
```

```
Operation: Pop()
Popped Element: 78
Stack: [50] [25] [ ] [ ] [ ] [ ] [ ]
           top=1
Empty Locations: 5
```

```
Operation: Pop()
Popped Element: 25
Stack: [50] [ ] [ ] [ ] [ ] [ ] [ ]
       top=0
Empty Locations: 6
```

```
Operation: Pop()
Popped Element: 50
Stack: [ ] [ ] [ ] [ ] [ ] [ ] [ ]
      top=-1 (Empty)
Empty Locations: 7
```

6. Complete Stack Implementation in C

```c
```

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100

typedef struct {
    int items[MAX_SIZE];
    int top;
} Stack;

// Initialize stack
void initStack(Stack *s) {
    s->top = -1;
}

// Check if empty
int isEmpty(Stack *s) {
    return (s->top == -1);
}

// Check if full
int isFull(Stack *s) {
    return (s->top == MAX_SIZE - 1);
}

// Push operation
void push(Stack *s, int item) {
    if (isFull(s)) {
        printf("Stack Overflow\n");
        return;
    }
    s->items[++(s->top)] = item;
    printf("Pushed %d\n", item);
}

// Pop operation
int pop(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow\n");
        return -1;
    }
    return s->items[(s->top)--];
}

// Peek/Top operation
int peek(Stack *s) {
    if (isEmpty(s)) {
```

```
        printf("Stack is empty\n");
        return -1;
    }
    return s->items[s->top];
}

// Display stack
void display(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack elements: ");
    for (int i = 0; i <= s->top; i++) {
        printf("%d ", s->items[i]);
    }
    printf("\n");
}
```

7. Advantages and Limitations

Advantages:

- Simple implementation and concept
- Constant time O(1) operations
- Memory efficient for sequential access
- Useful for function call management
- Natural fit for recursive algorithms

Limitations:

- Fixed size (in array implementation)
- No random access to elements
- Stack overflow risk with fixed size
- Memory wastage if stack size underutilized
- Cannot grow dynamically (in array version)

Memory Considerations:

- Total Memory = MAX_SIZE × sizeof(element) + sizeof(top)
- Example: For 100 integers: 100×4 + 4 = 404 bytes

8. Applications Preview

Stack data structure has numerous applications in computer science:

- Function call management and recursion
- Expression evaluation and conversion
- Syntax parsing in compilers
- Backtracking algorithms
- Undo mechanisms in applications
- Browser history navigation
- Memory management

Q2: Explain in detail the applications of stacks in expression conversion (Infix to Postfix, Infix to Prefix, and other conversions) with algorithms, examples, and operator precedence handling.

Answer:

1. Understanding Expression Notations

1.1 Types of Expressions

Infix Expression: Operator is placed between operands

- Format: `operand1 operator operand2`
- Example: `A + B`, `(A + B) * C`
- This is the standard mathematical notation humans use

Postfix Expression (Reverse Polish Notation):

- Format: `operand1 operand2 operator`
- Example: `A B +`, `A B + C *`
- No parentheses needed
- Operators follow their operands

Prefix Expression (Polish Notation):

- Format: `operator operand1 operand2`
- Example: `+ A B`, `* + A B C`
- No parentheses needed
- Operators precede their operands

1.2 Why Convert from Infix?

Problems with Infix Notation:

1. Requires precedence rules to evaluate
2. Needs associativity rules
3. Requires parentheses for clarity
4. Ambiguous without proper rules
5. Complex for computer evaluation

Example of Ambiguity:

- Expression: `3 + 5 * 4`
- Could mean: `(3 + 5) * 4 = 32` OR `3 + (5 * 4) = 23`
- Precedence rules dictate: `3 + (5 * 4) = 23`

Advantages of Postfix/Prefix:

- No ambiguity in evaluation order
- No parentheses required
- Easy to evaluate using stack
- Directly implementable by computers
- Single left-to-right scan sufficient

2. Operator Precedence and Associativity

2.1 Operator Precedence Table

```
Priority | Operators | Associativity
---------|-----------|---------------
  1      |     ^     | Right to Left
  2      |  *, /, %  | Left to Right
  3      |   +, -    | Left to Right
```

Precedence Rules:

- Higher precedence operators are evaluated first
- `^` (exponentiation) has highest precedence
- `*`, `/`, `%` have equal precedence (middle)
- `+`, `-` have lowest precedence
- Parentheses `()` override all precedence

2.2 In-Stack Priority (ISP) vs Incoming Priority (ICP)

For Infix to Postfix Conversion:

```
Operator | ICP (Incoming) | ISP (In-Stack)
---------|----------------|---------------
   (     |       5        |       0
   ^     |       4        |       3
  *, /   |       2        |       2
  +, -   |       1        |       1
```

For Infix to Prefix Conversion:

```
Operator | ICP (Incoming) | ISP (In-Stack)
---------|----------------|---------------
   (     |       5        |       0
   ^     |       3        |       4  (Swapped)
  *, /   |       2        |       2
  +, -   |       1        |       1
```

Note: For prefix conversion, ICP and ISP values for exponentiation (^) are swapped to handle right-to-left associativity.

3. Infix to Postfix Conversion

3.1 Algorithm

```
Algorithm InfixToPostfix(inexp[])
{
   k = 0; i = 0;
   tkn = inexp[i];

   while (tkn != '\0')
   {
      if (tkn is an operand)
      {
         postexp[k] = inexp[i];
         k++;
      }
      else  // token is operator or parenthesis
      {
         if (tkn == '(')  // opening parenthesis
         {
            push('(');
         }
         else if (tkn == ')')  // closing parenthesis
         {
            while ((tkn = pop()) != '(')
            {
               postexp[k] = tkn;
               k++;
            }
         }
         else  // operator
         {
            while (stack not empty &&
                  isp(stk[top]) >= icp(tkn))
            {
               postexp[k] = pop();
               k++;
            }
            push(tkn);
         }
      }
      // Read next token
      i++;
      tkn = inexp[i];
   }

   // Pop remaining operators
   while (stack not empty)
   {
      postexp[k] = pop();
```

```
        k++;
    }
}
```

3.2 Conversion Rules

1. **If operand**: Append directly to output
2. **If '('**: Push onto stack
3. **If ')'**: Pop and append to output until '(' is found; discard both parentheses
4. **If operator**:
    - While (stack not empty AND top operator precedence >= current operator precedence)
        - Pop and append to output
    - Push current operator onto stack
5. **At end**: Pop all remaining operators and append to output

3.3 Detailed Example 1: A * (B + C) - D / E

Step-by-Step Conversion:

```
Step 1: Initial State
Infix:    A * (B + C) - D / E
Stack:    Empty
Postfix:  Empty
```

```
Step 2: Read 'A' (Operand)
Action:   Append to output
Stack:    Empty
Postfix:  A
```

```
Step 3: Read '*' (Operator)
Action:   Stack empty, push *
Stack:    *
Postfix:  A
```

```
Step 4: Read '(' (Open parenthesis)
Action:   Push onto stack
Stack:    * (
Postfix:  A
```

Step 5: Read 'B' (Operand)
Action: Append to output
Stack: * (
Postfix: A B

Step 6: Read '+' (Operator)
Action: ICP(+)=1 > ISP(()=0, push +
Stack: * (+
Postfix: A B

Step 7: Read 'C' (Operand)
Action: Append to output
Stack: * (+
Postfix: A B C

Step 8: Read ')' (Close parenthesis)
Action: Pop until '(' found
Pop '+', append to output
Discard '(' and ')'
Stack: *
Postfix: A B C +

Step 9: Read '-' (Operator)
Action: ISP(*)=2 >= ICP(-)=1
Pop '*', append to output
Push '-'
Stack: -
Postfix: A B C + *

Step 10: Read 'D' (Operand)
Action: Append to output
Stack: -
Postfix: A B C + * D

Step 11: Read '/' (Operator)
Action: ICP(/)=2 > ISP(-)=1, push /
Stack: - /
Postfix: A B C + * D

```
Step 12: Read 'E' (Operand)
Action:   Append to output
Stack:    - /
Postfix:  A B C + * D E
```

```
Step 13: End of expression
Action:   Pop all operators
          Pop '/', append to output
          Pop '-', append to output
Stack:    Empty
Postfix:  A B C + * D E / -
```

Final Answer: `A B C + * D E / -`

3.4 Example 2: (a + b - c) * d - (e + f)

Conversion Table:

```
Token  | Action            | Stack  | Output
-------|-------------------|--------|-----------------
(      | Push              | (      |
a      | Append            | (      | a
+      | Push              | ( +    | a
b      | Append            | ( +    | a b
-      | Pop +, Push -     | ( -    | a b +
c      | Append            | ( -    | a b + c
)      | Pop until (       |        | a b + c -
*      | Push              | *      | a b + c -
d      | Append            | *      | a b + c - d
-      | Pop *, Push -     | -      | a b + c - d *
(      | Push              | - (    | a b + c - d *
e      | Append            | - (    | a b + c - d * e
+      | Push              | - ( +  | a b + c - d * e
f      | Append            | - ( +  | a b + c - d * e f
)      | Pop until (       | -      | a b + c - d * e f +
End    | Pop all           |        | a b + c - d * e f + -
```

Final Answer: `a b + c - d * e f + -`

3.5 Example 3: 2 * 3 / (2 - 1) + 5 * 3

Conversion Steps:

```
Infix Token | Stack State    | Postfix Output
------------|----------------|-------------------------
2           | Empty          | 2
*           | *              | 2
3           | *              | 2 3
/           | /              | 2 3 *
(           | / (            | 2 3 *
2           | / (            | 2 3 * 2
-           | / ( -          | 2 3 * 2
1           | / ( -          | 2 3 * 2 1
)           | /              | 2 3 * 2 1 -
+           | +              | 2 3 * 2 1 - /
5           | +              | 2 3 * 2 1 - / 5
*           | + *            | 2 3 * 2 1 - / 5
3           | + *            | 2 3 * 2 1 - / 5 3
End         | Empty          | 2 3 * 2 1 - / 5 3 * +
```

Final Answer: `2 3 * 2 1 - / 5 3 * +`

4. Infix to Prefix Conversion

4.1 Algorithm Steps

1. **Reverse** the infix expression
2. **Interchange** '(' with ')' and ')' with '('
3. Apply **modified infix to postfix** algorithm (with swapped ICP/ISP for ^)
4. **Reverse** the result to get prefix expression

4.2 Modified Algorithm

```
Algorithm InfixToPrefix(inexp[])
{
   // Step 1: Reverse the expression
   reverse(inexp);

   // Step 2: Swap parentheses
   for each character in inexp
      if char == '(' then char = ')'
      else if char == ')' then char = '('

   // Step 3: Convert using modified rules
   k = 0; i = 0;
   tkn = inexp[i];

   while (tkn != '\0')
   {
      if (tkn is operand)
      {
         pre_exp[k] = inexp[i];
         k++;
      }
      else
      {
         if (tkn == '(')
         {
            push('(');
         }
         else if (tkn == ')')
         {
            while ((tkn = pop()) != '(')
            {
               pre_exp[k] = tkn;
               k++;
            }
         }
         else  // operator
         {
            // Note: Use > instead of >=
            while (stack not empty &&
                  isp(stk[top]) > icp(tkn))
            {
               pre_exp[k] = pop();
               k++;
            }
            push(tkn);
         }
```

```
        }
        i++;
        tkn = inexp[i];
    }

    while (stack not empty)
    {
        pre_exp[k] = pop();
        k++;
    }

    // Step 4: Reverse the result
    reverse(pre_exp);
}
```

4.3 Detailed Example: (A + B ^ C) * D + E ^ 5

Step 1: Reverse the infix expression

```
Original:  (A + B ^ C) * D + E ^ 5
Reversed:  5 ^ E + D * )C ^ B + A(
```

Step 2: Swap parentheses

```
After swap: 5 ^ E + D * (C ^ B + A)
```

Step 3: Apply conversion algorithm

```
Token | Action           | Stack    | Output
------|------------------|----------|-------------
5     | Append           |          | 5
^     | Push             | ^        | 5
E     | Append           | ^        | 5 E
+     | Pop ^, Push +    | +        | 5 E ^
D     | Append           | +        | 5 E ^ D
*     | Push             | + *      | 5 E ^ D
(     | Push             | + * (    | 5 E ^ D
C     | Append           | + * (    | 5 E ^ D C
^     | Push             | + * ( ^  | 5 E ^ D C
B     | Append           | + * ( ^  | 5 E ^ D C B
+     | Pop ^, Push +    | + * ( +  | 5 E ^ D C B ^
A     | Append           | + * ( +  | 5 E ^ D C B ^ A
)     | Pop until (      | + *      | 5 E ^ D C B ^ A +
End   | Pop all          |          | 5 E ^ D C B ^ A + * +
```

Step 4: Reverse the output

```
Reversed: + + * + A ^ B C D ^ E 5
```

Final Answer: `+ + * + A ^ B C D ^ E 5`

Note: This seems incorrect. Let me recalculate...

Correct Step-by-Step:

```
Original Expression: (A + B ^ C) * D + E ^ 5

Step 1: Reverse
5 ^ E + D * ) C ^ B + A (

Step 2: Swap parentheses
5 ^ E + D * ( C ^ B + A )

Step 3: Convert (like postfix but with modified precedence)
Token | Stack    | Output
5     |          | 5
^     | ^        | 5
E     | ^        | 5 E
+     | +        | 5 E ^
D     | +        | 5 E ^ D
*     | + *      | 5 E ^ D
(     | + * (    | 5 E ^ D
C     | + * (    | 5 E ^ D C
^     | + * ( ^  | 5 E ^ D C
B     | + * ( ^  | 5 E ^ D C B
+     | + * ( +  | 5 E ^ D C B ^
A     | + * ( +  | 5 E ^ D C B ^ A
)     | + *      | 5 E ^ D C B ^ A +
End   |          | 5 E ^ D C B ^ A + * +

Step 4: Reverse
+ * + A ^ B C D ^ E 5
```

Actually the correct answer is: `+ * + A ^ B C D ^ E 5`

5. Other Expression Conversions

5.1 Postfix to Infix Conversion

Algorithm:

```
1. Read symbol from postfix expression
2. If symbol is operand:
   - Push it onto stack
3. If symbol is operator:
   - Pop top 2 values (operand2, operand1)
   - Create string: (operand1 operator operand2)
   - Push this string back onto stack
4. At end, stack contains one element - the infix expression
```

Example: `A B + C * D - E F + -`

```
Symbol | Operator | Opnd1  | Opnd2  | Stack
-------|----------|--------|--------|----------------
A      |          |        |        | A
B      |          |        |        | A, B
+      | +        | A      | B      | (A+B)
C      |          |        |        | (A+B), C
*      | *        | (A+B)  | C      | ((A+B)*C)
D      |          |        |        | ((A+B)*C), D
-      | -        |(A+B)*C | D      | (((A+B)*C)-D)
E      |          |        |        | (((A+B)*C)-D), E
F      |          |        |        | (((A+B)*C)-D), E, F
+      | +        | E      | F      | (((A+B)*C)-D), (E+F)
-      | -        |(A+B)*C-D|(E+F)  | ((((A+B)*C)-D)-(E+F))
```

Final Answer: `((((A+B)*C)-D)-(E+F))` or simplified as `((A+B)*C-D-(E+F))`

5.2 Postfix to Prefix Conversion

Algorithm:

```
1. Read postfix expression from LEFT to RIGHT (reverse order)
2. If symbol is operand:
   - Push onto stack
3. If symbol is operator:
   - Pop 2 operands (op1, op2)
   - Create: operator + op1 + op2
   - Push back onto stack
4. Result at top of stack
```

Example: `A B C / - A K / L - *`

Reading from right to left: `* - / L K A - / C B A`

```
Symbol | Operator | Op1    | Op2    | Stack
-------|----------|--------|--------|-----------------
*      |          |        |        | *
-      |          |        |        | *, -
/      |          |        |        | *, -, /
L      |          |        |        | *, -, /, L
K      |          |        |        | *, -, /, L, K
A      |          |        |        | *, -, /, L, K, A
-      | -        | K      | A      | *, -, /, L, -KA
/      | /        | A      | K      | *, -, /, L, /AK
L      |          |        |        | *, -, /, L, /AK, L
-      | -        | /AK    | L      | *, -, /, -/AKL
/      | /        | C      | B      | *, -, /, -/AKL, /CB
C      |          |        |        | *, -, /, -/AKL, /CB, C
B      |          |        |        | *, -, /, -/AKL, /CB, C, B
A      |          |        |        | *, -, /, -/AKL, /CB, C, B, A
-      | -        | /CB    | A      | *, -, -A/BC
*      | *        | -A/BC  | -/AKL  | *-A/BC-/AKL
```

Final Answer: `*-A/BC-/AKL`

5.3 Prefix to Postfix Conversion

Algorithm:

```
1. Read prefix expression from RIGHT to LEFT
2. If symbol is operand:
   - Push onto stack
3. If symbol is operator:
   - Pop 2 operands (op1, op2)
   - Create: op1 + op2 + operator
   - Push back onto stack
4. Result at top of stack
```

Example: `* - A / B C - / A K L`

Reading from right to left: `L K A / - C B / A - *`

```
Symbol | Op    | Op1  | Op2    | Stack
-------|-------|------|--------|-----------------
L      |       |      |        | L
K      |       |      |        | L, K
A      |       |      |        | L, K, A
/      | /     | A    | K      | L, AK/
-      | -     | AK/  | L      | AK/L-
C      |       |      |        | AK/L-, C
B      |       |      |        | AK/L-, C, B
/      | /     | B    | C      | AK/L-, BC/
A      |       |      |        | AK/L-, BC/, A
-      | -     | A    | BC/    | AK/L-, ABC/-
*      | *     | ABC/-| AK/L-  | ABC/-AK/L-*
```

Final Answer: `ABC/-AK/L-*`

5.4 Prefix to Infix Conversion

Algorithm:

```
1. Read prefix expression from RIGHT to LEFT
2. If symbol is operand:
   - Push onto stack
3. If symbol is operator:
   - Pop 2 operands (op1, op2)
   - Create: (op1 operator op2)
   - Push back onto stack
4. Result at top of stack
```

Example: `*-A/BC-/AKL`

Reading right to left: `LKA/-CB/A-*`

```
Symbol | Op | Op1      | Op2    | Stack
-------|----| ---------|--------|----------------
L      |    |          |        | L
K      |    |          |        | L, K
A      |    |          |        | L, K, A
/      | / | A        | K      | L, (A/K)
-      | - | (A/K)    | L      | ((A/K)-L)
C      |    |          |        | ((A/K)-L), C
B      |    |          |        | ((A/K)-L), C, B
/      | / | B        | C      | ((A/K)-L), (B/C)
A      |    |          |        | ((A/K)-L), (B/C), A
-      | - | A        | (B/C)  | ((A/K)-L), (A-(B/C))
*      | * |(A-(B/C)) |((A/K)-L)| ((A-(B/C))*((A/K)-L))
```

Final Answer: `((A-(B/C))*((A/K)-L))`

6. Summary of Conversion Rules

Quick Reference Table:

From → To	Method
Infix → Postfix	Use stack, scan left-to-right, output operands immediately, handle operators by precedence
Infix → Prefix	Reverse infix, swap parentheses, apply modified postfix algorithm, reverse result
Postfix → Infix	Scan left-to-right, push operands, for operators: pop 2, form (op1 op op2), push back
Postfix → Prefix	Scan left-to-right, push operands, for operators: pop 2, form (op op1 op2), push back
Prefix → Postfix	Scan right-to-left, push operands, for operators: pop 2, form (op1 op2 op), push back
Prefix → Infix	Scan right-to-left, push operands, for operators: pop 2, form (op1 op op2), push back

7. Time and Space Complexity

All Conversions:

- **Time Complexity**: O(n) where n is the number of tokens
- **Space Complexity**: O(n) for stack storage

8. Practical Implementation Considerations

Handling Multi-character Operands:

- Use delimiters or special markers
- Example: `AB+` could mean A+B or (A*B)+?
- Solution: `A B +` with spaces

Error Handling:

- Mismatched parentheses
- Invalid operator sequences
- Stack overflow/underflow
- Empty expressions

Q3: Explain expression evaluation using stacks for postfix and prefix expressions with detailed algorithms, examples, and step-by-step calculations.

Answer:

1. Introduction to Expression Evaluation

After converting expressions to postfix or prefix notation, the next step is evaluation. Stack-based evaluation is efficient because postfix and prefix notations eliminate the need for precedence rules and parentheses during computation.

Why Postfix/Prefix for Evaluation?

- No need to check operator precedence
- No parentheses to handle
- Single left-to-right (or right-to-left) scan
- Direct stack-based implementation
- O(n) time complexity

2. Postfix Expression Evaluation

2.1 Algorithm

```
Algorithm EvaluatePostfix(postfix[])
{
   Initialize an empty stack (operand stack)

   for each symbol in postfix expression (left to right)
   {
      if (symbol is operand)
      {
         push(symbol onto stack)
      }
      else if (symbol is operator)
      {
         opnd2 = pop()    // Second operand
         opnd1 = pop()    // First operand

         result = apply operator to (opnd1, opnd2)

         push(result onto stack)
      }
   }

   final_result = pop()
   return final_result
}
```

Key Points:

- Operands are pushed onto stack
- When operator encountered, pop 2 operands
- **Order matters**: First popped is second operand
- Apply operator: result = opnd1 operator opnd2
- Push result back
- Final stack element is answer

2.2 Detailed Example 1: 6 2 3 + - 3 8 2 / + * 2 ^ 3 +

Given: Postfix expression = `6 2 3 + - 3 8 2 / + * 2 ^ 3 +`

Step-by-Step Evaluation:

```
Symbol | Op   | Opnd1 | Opnd2 | Value | Stack
-------|------|-------|-------|-------|-----------------
6      |      |       |       |       | 6
2      |      |       |       |       | 6, 2
3      |      |       |       |       | 6, 2, 3
+      | +    | 2     | 3     | 5     | 6, 5
-      | -    | 6     | 5     | 1     | 1
3      |      |       |       |       | 1, 3
8      |      |       |       |       | 1, 3, 8
2      |      |       |       |       | 1, 3, 8, 2
/      | /    | 8     | 2     | 4     | 1, 3, 4
+      | +    | 3     | 4     | 7     | 1, 7
*      | *    | 1     | 7     | 7     | 7
2      |      |       |       |       | 7, 2
^      | ^    | 7     | 2     | 49    | 49
3      |      |       |       |       | 49, 3
+      | +    | 49    | 3     | 52    | 52
```

Final Answer: 52

Calculation Verification:

```
Original expression (reconstructed):
((6 - (2 + 3)) * (3 + (8 / 2))) ^ 2 + 3
= ((6 - 5) * (3 + 4)) ^ 2 + 3
= (1 * 7) ^ 2 + 3
= 7 ^ 2 + 3
= 49 + 3
= 52
```

2.3 Example 2: A B + C * D - E F * G / +

Given Values: A=1, B=2, C=4, D=5, E=3, F=2, G=10

```
Token | Op | Op1 | Op2 | Value | Stack
------|----| ----|-----|-------|-----------------
1     |    |     |     |       | 1
2     |    |     |     |       | 1, 2
+     | +  | 1   | 2   | 3     | 3
4     |    |     |     |       | 3, 4
*     | *  | 3   | 4   | 12    | 12
5     |    |     |     |       | 12, 5
-     | -  | 12  | 5   | 7     | 7
3     |    |     |     |       | 7, 3
2     |    |     |     |       | 7, 3, 2
*     | *  | 3   | 2   | 6     | 7, 6
10    |    |     |     |       | 7, 6, 10
/     | /  | 6   | 10  | 0.6   | 7, 0.6
+     | +  | 7   | 0.6 | 7.6   | 7.6
```

Final Answer: 7.6

Calculation:

```
= ((1+2)*4 - 5) + (3*2)/10
= (3*4 - 5) + 6/10
= (12 - 5) + 0.6
= 7 + 0.6
= 7.6
```

2.4 Example 3: A B C + * C B A - + *

Given Values: A=10, B=2, C=13

```
Token | Op | Op1 | Op2 | Value | Stack
------|----| ----|-----|-------|-----------------
10    |    |     |     |       | 10
2     |    |     |     |       | 10, 2
13    |    |     |     |       | 10, 2, 13
+     | +  | 2   | 13  | 15    | 10, 15
*     | *  | 10  | 15  | 150   | 150
13    |    |     |     |       | 150, 13
2     |    |     |     |       | 150, 13, 2
10    |    |     |     |       | 150, 13, 2, 10
-     | -  | 2   | 10  | -8    | 150, 13, -8
+     | +  | 13  | -8  | 5     | 150, 5
*     | *  | 150 | 5   | 750   | 750
```

Final Answer: 750

2.5 C Implementation

```c
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define MAX 100

typedef struct {
    int items[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void push(Stack *s, int value) {
    if (s->top < MAX - 1) {
        s->items[++(s->top)] = value;
    }
}

int pop(Stack *s) {
    if (!isEmpty(s)) {
        return s->items[(s->top)--];
    }
    return -1;
}

int evaluatePostfix(char* exp) {
    Stack s;
    initStack(&s);

    for (int i = 0; exp[i]; i++) {
        // Skip whitespace
        if (exp[i] == ' ') continue;

        // If operand, push to stack
        if (isdigit(exp[i])) {
            int num = 0;
            while (isdigit(exp[i])) {
                num = num * 10 + (exp[i] - '0');
```

```
            i++;
        }
        i--; // Adjust for loop increment
        push(&s, num);
    }
    // If operator, pop two operands and apply
    else {
        int op2 = pop(&s);
        int op1 = pop(&s);

        switch(exp[i]) {
            case '+': push(&s, op1 + op2); break;
            case '-': push(&s, op1 - op2); break;
            case '*': push(&s, op1 * op2); break;
            case '/': push(&s, op1 / op2); break;
            case '^': push(&s, pow(op1, op2)); break;
        }
    }
  }

  return pop(&s);
}

int main() {
  char exp[] = "6 2 3 + - 3 8 2 / + * 2 ^ 3 +";
  printf("Result: %d\n", evaluatePostfix(exp));
  return 0;
}
```

3. Prefix Expression Evaluation

3.1 Algorithm

```
Algorithm EvaluatePrefix(prefix[])
{
   Initialize an empty stack

   // Scan from RIGHT to LEFT
   for i = length-1 down to 0
   {
      symbol = prefix[i]

      if (symbol is operand)
      {
         push(symbol onto stack)
      }
      else if (symbol is operator)
      {
         opnd1 = pop()    // First operand
         opnd2 = pop()    // Second operand

         result = apply operator to (opnd1, opnd2)

         push(result onto stack)
      }
   }

   final_result = pop()
   return final_result
}
```

Key Differences from Postfix:

- Scan from **RIGHT to LEFT**
- After popping, order is: opnd1 operator opnd2 (not reversed)
- Everything else same as postfix

3.2 Detailed Example: - + * 2 3 * 5 4 9

Expression: `- + * 2 3 * 5 4 9`

Reading RIGHT to LEFT: `9 4 5 * 3 2 * + -`

```
Symbol | Op | Op1 | Op2 | Value | Stack
-------|----| ----|-----|-------|-----------------
9      |    |     |     |       | 9
4      |    |     |     |       | 9, 4
5      |    |     |     |       | 9, 4, 5
*      | *  | 4   | 5   | 20    | 9, 20
3      |    |     |     |       | 9, 20, 3
2      |    |     |     |       | 9, 20, 3, 2
*      | *  | 3   | 2   | 6     | 9, 20, 6
+      | +  | 20  | 6   | 26    | 9, 26
-      | -  | 9   | 26  | -17   | -17
```

Wait, this doesn't look right. Let me recalculate...

Actually, when scanning prefix from right to left and we encounter an operator:

- Pop first operand (op1)
- Pop second operand (op2)
- Calculate: op1 operator op2

```
Symbol | Op | Op1 | Op2 | Calc     | Value | Stack
-------|----| ----|-----|----------|-------|----------
9      |    |     |     |          |       | 9
4      |    |     |     |          |       | 9, 4
5      |    |     |     |          |       | 9, 4, 5
*      | *  | 4   | 5   | 4 * 5    | 20    | 9, 20
3      |    |     |     |          |       | 9, 20, 3
2      |    |     |     |          |       | 9, 20, 3, 2
*      | *  | 3   | 2   | 3 * 2    | 6     | 9, 20, 6
+      | +  | 20  | 6   | 20 + 6   | 26    | 9, 26
-      | -  | 9   | 26  | 9 - 26   | -17   | -17
```

Hmm, let me verify with the original expression:

```
- + * 2 3 * 5 4 9
= - (+ (* 2 3) (* 5 4)) 9
= - (+ 6 20) 9
= - 26 9
= 26 - 9
= 17
```

So the issue is operator application. For prefix, when scanning right-to-left:

- **Result = op1 operator op2** where op1 is first popped

But actually for subtraction: - (something) 9 means something - 9, not 9 - something

Let me reconsider: when we see `-` as the last operator and pop 9 then 26:

```
The prefix is: - + ... 9
This means: (+ ...) - 9
So: result = 26 - 9 = 17 (not 9 - 26)
```

Corrected calculation:

```
Symbol | Op | Op1 | Op2 | Calculation | Value | Stack
-------|----| ----|-----|-------------|-------|-------
9      |    |     |     |             |       | 9
4      |    |     |     |             |       | 9, 4
5      |    |     |     |             |       | 9, 4, 5
*      | * | 4   | 5   | 4 * 5       | 20    | 9, 20
3      |    |     |     |             |       | 9, 20, 3
2      |    |     |     |             |       | 9, 20, 3, 2
*      | * | 3   | 2   | 3 * 2       | 6     | 9, 20, 6
+      | + | 20  | 6   | 20 + 6      | 26    | 9, 26
-      | - | 9   | 26  | 26 - 9      | 17    | 17
```

Wait, that's also wrong. When scanning right-to-left in prefix and we pop operands:

- First pop gives us the rightmost operand in the original prefix notation
- For `- something 9`: we want (something - 9)
- When we scan RTL and see `-`, we've already pushed 9
- So first pop gives 9, second pop gives (something)
- We need: second_pop - first_pop = something - 9

Actually Correct:

For prefix evaluation scanning RIGHT to LEFT:

```
result = op2 operator op1
```

Where op1 is first popped, op2 is second popped.

```
Symbol | Op | Op1 | Op2 | Calculation | Value | Stack
-------|----| ----|-----|-------------|-------|-------
9      |    |     |     |             |       | 9
4      |    |     |     |             |       | 9, 4
5      |    |     |     |             |       | 9, 4, 5
*      | *  | 5   | 4   | 4 * 5       | 20    | 9, 20
3      |    |     |     |             |       | 9, 20, 3
2      |    |     |     |             |       | 9, 20, 3, 2
*      | *  | 2   | 3   | 3 * 2       | 6     | 9, 20, 6
+      | +  | 6   | 20  | 20 + 6      | 26    | 9, 26
-      | -  | 26  | 9   | 9 - 26      | -17   | -17
```

Actually no! Let's think carefully:

Prefix: `- + * 2 3 * 5 4 9` Means: `(+ (* 2 3) (* 5 4)) - 9` = `(+ 6 20) - 9` = `26 - 9` = `17`

When scanning RTL and we encounter `-`:

- We've already pushed everything to the right of `-`
- Stack has: [9, 26]
- Pop once: gets 26 (this is the left operand of -)
- Pop twice: gets 9 (this is the right operand of -)
- We want: 26 - 9
- So: **result = op1 - op2** where op1 is first popped

```
Symbol | Op | Op1 | Op2 | Calculation | Value | Stack
-------|----| ----|-----|-------------|-------|-------
9      |    |     |     |             |       | 9
4      |    |     |     |             |       | 9, 4
5      |    |     |     |             |       | 9, 4, 5
*      | *  | 5   | 4   | 5 * 4       | 20    | 9, 20
3      |    |     |     |             |       | 9, 20, 3
2      |    |     |     |             |       | 9, 20, 3, 2
*      | *  | 2   | 3   | 2 * 3       | 6     | 9, 20, 6
+      | +  | 6   | 20  | 6 + 20      | 26    | 9, 26
-      | -  | 26  | 9   | 26 - 9      | 17    | 17
```

Final Answer: **17**

3.3 Another Example: + - * 2 3 / 6 3 1

Expression: `+ - * 2 3 / 6 3 1`

Reading RIGHT to LEFT: `1 3 6 / 3 2 * - +`

```
Symbol | Op | Op1 | Op2 | Calculation  | Value | Stack
-------|----| ----|-----|--------------|-------|-------
1      |    |     |     |              |       | 1
3      |    |     |     |              |       | 1, 3
6      |    |     |     |              |       | 1, 3, 6
/      | / | 6   | 3   | 6 / 3        | 2     | 1, 2
3      |    |     |     |              |       | 1, 2, 3
2      |    |     |     |              |       | 1, 2, 3, 2
*      | * | 2   | 3   | 2 * 3        | 6     | 1, 2, 6
-      | - | 6   | 2   | 6 - 2        | 4     | 1, 4
+      | + | 4   | 1   | 4 + 1        | 5     | 5
```

Final Answer: 5

Verification:

```
+ - * 2 3 / 6 3 1
= (- (* 2 3) (/ 6 3)) + 1
= (- 6 2) + 1
= 4 + 1
= 5
```

4. Comparison: Postfix vs Prefix Evaluation

Aspect	**Postfix**	**Prefix**
Scan Direction	Left to Right	Right to Left
When Operand	Push to stack	Push to stack
When Operator	Pop 2, calculate	Pop 2, calculate
Operator Application	op1 operator op2	op1 operator op2
Complexity	O(n)	O(n)
Space	O(n)	O(n)

Note: In both cases, op1 is first popped, op2 is second popped.

5. Complete C Implementation for Prefix

```c
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include <math.h>

#define MAX 100

typedef struct {
    int items[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void push(Stack *s, int value) {
    s->items[++(s->top)] = value;
}

int pop(Stack *s) {
    return s->items[(s->top)--];
}

int evaluatePrefix(char* exp) {
    Stack s;
    initStack(&s);

    int len = strlen(exp);

    // Scan from right to left
    for (int i = len - 1; i >= 0; i--) {
        // Skip whitespace
        if (exp[i] == ' ') continue;

        // If operand
        if (isdigit(exp[i])) {
            int num = 0;
            int multiplier = 1;

            while (i >= 0 && isdigit(exp[i])) {
```

```
            num = (exp[i] - '0') * multiplier + num;
            multiplier *= 10;
            i--;
        }
        i++; // Adjust
        push(&s, num);
    }
    // If operator
    else {
        int op1 = pop(&s);
        int op2 = pop(&s);

        switch(exp[i]) {
            case '+': push(&s, op1 + op2); break;
            case '-': push(&s, op1 - op2); break;
            case '*': push(&s, op1 * op2); break;
            case '/': push(&s, op1 / op2); break;
            case '^': push(&s, pow(op1, op2)); break;
        }
    }
  }

  return pop(&s);
}
```

6. Time and Space Complexity Analysis

Both Postfix and Prefix Evaluation:

Time Complexity: O(n)

- Single pass through expression
- Each symbol processed once
- Stack operations are O(1)

Space Complexity: O(n)

- Stack can grow up to n/2 elements (worst case)
- Example: `1 2 3 4 5 + + + +` pushes 5 elements before any operation

Best Case: O(1) space - when operators appear frequently **Worst Case**: O(n) space - when all operands come first

7. Error Handling and Edge Cases

Common Errors:

1. **Insufficient Operands**: `2 + 3` (postfix) - missing second operand
2. **Too Many Operands**: `2 3 4 +` - extra operand left
3. **Division by Zero**: `5 0 /`
4. **Invalid Characters**: Non-operator, non-operand symbols
5. **Empty Expression**: No tokens to process

Robust Implementation Should Check:

- Stack underflow during pop
- Final stack should have exactly one element
- Valid operators only
- Numeric operands only (or valid variables)

8. Practical Applications

Expression Evaluation Used In:

- Calculators and computing devices
- Compiler expression evaluation
- Spreadsheet formula calculation
- Database query optimization
- Mathematical software
- Programming language interpreters

Q4: Explain recursion in detail, including its relationship with stacks, implementation examples (factorial, Fibonacci), advantages, disadvantages, and comparison with iteration.

Answer:

1. Introduction to Recursion

Recursion is a programming technique where a function calls itself directly or indirectly to solve a problem by breaking it down into smaller, similar subproblems.

Definition: A function is recursive if it is defined in terms of itself.

Key Concept: "The best way to solve a problem is by solving a smaller version of the exact same problem first."

2. How Recursion Works

Basic Structure:

```
function recursive_function(parameters)
{
   if (base_case_condition)
   {
      return base_case_value;  // Stopping condition
   }
   else
   {
      // Recursive case
      return some_operation(recursive_function(modified_parameters));
   }
}
```

Essential Components:

1. **Base Case** (Terminating Condition):
   - Simplest version of the problem
   - Can be solved directly without recursion
   - Prevents infinite recursion
   - Must be reached eventually
2. **Recursive Case**:
   - Breaks problem into smaller subproblems
   - Calls the function itself with modified parameters
   - Moves towards the base case
3. **Progress Towards Base Case**:
   - Each recursive call must move closer to base case
   - Parameters must change in each call
   - Eventually base case must be reached

3. Recursion and Stack Relationship

How Recursion Uses Stack:

Every recursive call creates a new **activation record** (stack frame) containing:

- Local variables
- Parameters

- Return address
- Intermediate results

These frames are pushed onto the **system call stack** (runtime stack).

Stack Operations During Recursion:

```
Function Call → Push activation record
Function Return → Pop activation record
```

Memory Organization:

```
|--------------------|
| Stack (grows down) | ← Recursive calls stored here
|--------------------|
| Heap (grows up)    |
|--------------------|
| Static/Global Data |
|--------------------|
| Code/Text Segment  |
|--------------------|
```

4. Factorial Example - Detailed Analysis

Problem: Calculate n! = n × (n-1) × (n-2) × ... × 2 × 1

Recursive Definition:

```
factorial(n) = 1                  if n == 0 or n == 1  (base case)
             = n × factorial(n-1)  if n > 1            (recursive case)
```

Implementation:

```c
int factorial(int n) {
    if (n == 0 || n == 1)  // Base case
        return 1;
    else                   // Recursive case
        return n * factorial(n - 1);
}
```

Execution Trace for factorial(5):

```
Call Stack Visualization:

factorial(5)
├── 5 * factorial(4)
│   ├── 4 * factorial(3)
│   │   ├── 3 * factorial(2)
│   │   │   ├── 2 * factorial(1)
│   │   │   │   └── return 1
│   │   │   └── return 2 * 1 = 2
│   │   └── return 3 * 2 = 6
│   └── return 4 * 6 = 24
└── return 5 * 24 = 120
```

Detailed Stack Frames:

Step 1: factorial(5) called
Stack: [factorial(5): n=5, return to main]

Step 2: factorial(4) called from factorial(5)
Stack: [factorial(5): n=5, return to main]
    [factorial(4): n=4, return to L1 in factorial(5)]

Step 3: factorial(3) called from factorial(4)
Stack: [factorial(5): n=5, return to main]
    [factorial(4): n=4, return to L1 in factorial(5)]
    [factorial(3): n=3, return to L1 in factorial(4)]

Step 4: factorial(2) called from factorial(3)
Stack: [factorial(5): n=5, return to main]
    [factorial(4): n=4, return to L1 in factorial(5)]
    [factorial(3): n=3, return to L1 in factorial(4)]
    [factorial(2): n=2, return to L1 in factorial(3)]

Step 5: factorial(1) called from factorial(2)
Stack: [factorial(5): n=5, return to main]
    [factorial(4): n=4, return to L1 in factorial(5)]
    [factorial(3): n=3, return to L1 in factorial(4)]
    [factorial(2): n=2, return to L1 in factorial(3)]
    [factorial(1): n=1, return to L1 in factorial(2)]

Step 6: Base case reached, factorial(1) returns 1
Stack: [factorial(5): n=5, return to main]
    [factorial(4): n=4, return to L1 in factorial(5)]
    [factorial(3): n=3, return to L1 in factorial(4)]
    [factorial(2): n=2, return to L1 in factorial(3)]
    ← Pop, return 1

Step 7: factorial(2) = 2 * 1 = 2, returns
Stack: [factorial(5): n=5, return to main]
    [factorial(4): n=4, return to L1 in factorial(5)]
    [factorial(3): n=3, return to L1 in factorial(4)]
    ← Pop, return 2

Step 8: factorial(3) = 3 * 2 = 6, returns
Stack: [factorial(5): n=5, return to main]
    [factorial(4): n=4, return to L1 in factorial(5)]
    ← Pop, return 6

Step 9: factorial(4) = 4 * 6 = 24, returns
Stack: [factorial(5): n=5, return to main]
    ← Pop, return 24

```
Step 10: factorial(5) = 5 * 24 = 120, returns to main
Stack: Empty
Result: 120
```

Calculation Flow:

```
factorial(5)
= 5 * factorial(4)
= 5 * (4 * factorial(3))
= 5 * (4 * (3 * factorial(2)))
= 5 * (4 * (3 * (2 * factorial(1))))
= 5 * (4 * (3 * (2 * 1)))
= 5 * (4 * (3 * 2))
= 5 * (4 * 6)
= 5 * 24
= 120
```

5. Fibonacci Example

Problem: Find nth Fibonacci number where F(n) = F(n-1) + F(n-2)

Recursive Definition:

```
fib(n) = 0                    if n == 0  (base case)
       = 1                    if n == 1  (base case)
       = fib(n-1) + fib(n-2)  if n > 1   (recursive case)
```

Implementation:

c

```c
int fibonacci(int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Execution Tree for fibonacci(5):

Result Calculation:

```
fib(5) = fib(4) + fib(3)
       = (fib(3) + fib(2)) + (fib(2) + fib(1))
       = ((fib(2) + fib(1)) + (fib(1) + fib(0))) + ((fib(1) + fib(0)) + 1)
       = (((1 + 0) + 1) + (1 + 0)) + ((1 + 0) + 1)
       = ((1 + 1) + 1) + (1 + 1)
       = (2 + 1) + 2
       = 3 + 2
       = 5
```

Function Call Count:

- fib(5): 1 call
- fib(4): 1 call
- fib(3): 2 calls
- fib(2): 3 calls
- fib(1): 5 calls
- fib(0): 3 calls **Total**: 15 function calls for fib(5)

6. More Recursion Examples

6.1 Tower of Hanoi

Problem: Move n disks from source peg to destination peg using auxiliary peg.

Rules:

- Only one disk can be moved at a time
- Larger disk cannot be placed on smaller disk
- Only top disk of any peg can be moved

Recursive Solution:

```c
void towerOfHanoi(int n, char source, char dest, char aux) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", source, dest);
        return;
    }

    // Move n-1 disks from source to auxiliary using destination
    towerOfHanoi(n - 1, source, aux, dest);

    // Move nth disk from source to destination
    printf("Move disk %d from %c to %c\n", n, source, dest);

    // Move n-1 disks from auxiliary to destination using source
    towerOfHanoi(n - 1, aux, dest, source);
}
```

For n=3, Output:

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Number of moves: 2^n - 1 = 2^3 - 1 = 7 moves

6.2 Greatest Common Divisor (GCD)

Euclidean Algorithm:

```c
int gcd(int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd(b, a % b);
}
```

Example: gcd(48, 18)

```
gcd(48, 18) = gcd(18, 48 % 18) = gcd(18, 12)
            = gcd(12, 18 % 12) = gcd(12, 6)
            = gcd(6, 12 % 6)   = gcd(6, 0)
            = 6
```

6.3 Sum of Digits

```c
int sumOfDigits(int n) {
    if (n == 0)
        return 0;
    else
        return (n % 10) + sumOfDigits(n / 10);
}
```

Example: sumOfDigits(1234)

```
sumOfDigits(1234) = 4 + sumOfDigits(123)
                  = 4 + (3 + sumOfDigits(12))
                  = 4 + (3 + (2 + sumOfDigits(1)))
                  = 4 + (3 + (2 + (1 + sumOfDigits(0))))
                  = 4 + (3 + (2 + (1 + 0)))
                  = 4 + 3 + 2 + 1
                  = 10
```

7. Properties of Recursive Solutions

A Problem is Suitable for Recursion if:

1. **One or more stopping cases** have simple, non-recursive solutions
2. **Other cases** can be reduced using recursion to problems closer to stopping cases
3. **Eventually** the problem can be reduced to only stopping cases

Steps to Solve Recursive Problems:

1. **Express problem as simpler version of itself**
2. **Determine the stopping cases** (base cases)
3. **Determine the recursive steps**

General Recursive Template:

```
IF stopping case is reached
    SOLVE it directly
ELSE
    SPLIT the problem into simpler cases using recursion
    COMBINE solutions
```

8. Recursion vs Iteration

8.1 Comparison Table

Aspect	Recursion	Iteration
Definition	Function calls itself	Uses loops (for, while)
Termination	Base case	Loop condition
Memory	Uses call stack	Uses program stack
Space	Higher (O(n) for depth n)	Lower (O(1) typically)
Speed	Slower (function call overhead)	Faster
Code	Often shorter, cleaner	Can be longer
Readability	More intuitive for some problems	Can be complex
Stack Overflow	Risk if depth too large	No risk
Overhead	Function call overhead	No overhead

8.2 Factorial: Recursive vs Iterative

Recursive:

c

```c
int factorial_recursive(int n) {
    if (n <= 1)
        return 1;
    return n * factorial_recursive(n - 1);
}
```

Iterative:

c

```c
int factorial_iterative(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Comparison:

- Recursive: Clearer logic, but O(n) space
- Iterative: O(1) space, faster execution

8.3 Fibonacci: Recursive vs Iterative

Recursive (Inefficient):

```c
int fib_recursive(int n) {
    if (n <= 1)
        return n;
    return fib_recursive(n - 1) + fib_recursive(n - 2);
}
```

Time: O(2^n), **Space**: O(n)

Iterative (Efficient):

```c
int fib_iterative(int n) {
    if (n <= 1)
        return n;

    int prev = 0, curr = 1, next;
    for (int i = 2; i <= n; i++) {
        next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}
```

Time: O(n), **Space**: O(1)

9. Advantages of Recursion

1. **Simplicity**: Code is cleaner and easier to understand
2. **Natural Fit**: Some problems naturally recursive (tree traversal, divide-and-conquer)
3. **Reduced Code Length**: Often shorter than iterative solutions
4. **Problem Decomposition**: Breaks complex problems into simpler subproblems
5. **Mathematical Elegance**: Directly implements mathematical definitions

Best Suited For:

- Tree and graph traversals
- Divide and conquer algorithms
- Backtracking problems
- Mathematical sequences
- Dynamic programming (with memoization)

10. Disadvantages of Recursion

1. **Memory Overhead**: Each call consumes stack memory
2. **Slower Execution**: Function call overhead
3. **Stack Overflow**: Deep recursion can exhaust stack space
4. **Debugging Difficulty**: Harder to trace and debug
5. **Redundant Computation**: May recalculate same values (like Fibonacci)

Common Pitfalls:

c

```c
// Infinite recursion - no base case
void infinite() {
    printf("This will crash!\n");
    infinite();  // Stack overflow!
}

// Wrong base case
int wrong_factorial(int n) {
    if (n == 2)  // Base case never reached for n < 2
        return 2;
    return n * wrong_factorial(n - 1);
}
```

11. When to Use Recursion vs Iteration

Use Recursion When:

- Problem has natural recursive structure
- Tree/graph traversal required
- Code clarity is priority
- Depth is manageable
- Examples: Tree traversal, quicksort, mergesort

Use Iteration When:

- Performance is critical
- Memory is constrained
- Deep recursion expected
- Simple loop structure possible
- Examples: Array traversal, factorial, Fibonacci

Can Convert Any Recursion to Iteration:

- Use explicit stack data structure
- Maintain state manually
- Often results in more complex code

12. Tail Recursion Optimization

Tail Recursion: Recursive call is the last operation in the function.

Example:

```c
// Tail recursive factorial
int factorial_tail(int n, int accumulator) {
    if (n <= 1)
        return accumulator;
    return factorial_tail(n - 1, n * accumulator);
}

// Call: factorial_tail(5, 1)
```

Benefit: Compilers can optimize tail recursion to iteration, eliminating stack overhead.

Non-tail vs Tail:

```c
// Non-tail (operation after recursive call)
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);  // Multiplication after call
}

// Tail (nothing after recursive call)
int factorial_tail(int n, int acc) {
    if (n <= 1) return acc;
    return factorial_tail(n - 1, n * acc);  // Direct return
}
```

13. Recursion and Stack Overflow

Stack Overflow Error: Occurs when recursion depth exceeds stack size limit.

Example:

```c
// This will cause stack overflow for large n
int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n - 1);
}

// Calling sum(100000) will likely crash
```

Prevention:

1. Use iteration for deep recursion
2. Implement tail recursion
3. Use memoization for repeated calculations
4. Increase stack size (not recommended)

14. Recursion with Memoization

Problem: Naive Fibonacci is O(2^n) due to repeated calculations.

Solution: Store calculated values (memoization).

```c
```

```c
#define MAX 100
int memo[MAX];

int fib_memo(int n) {
    if (n <= 1)
        return n;

    if (memo[n] != -1)  // Already calculated
        return memo[n];

    memo[n] = fib_memo(n - 1) + fib_memo(n - 2);
    return memo[n];
}

// Initialize: for(int i=0; i<MAX; i++) memo[i] = -1;
```

Improvement: O(2^n) → O(n) time complexity

Q5: Explain linked stack implementation in detail, including structure definition, all operations (push, pop, isEmpty), advantages over array implementation, and complete working examples.

Answer:

1. Introduction to Linked Stack

A **linked stack** is a stack implementation using a singly linked list data structure. Unlike array-based stacks, linked stacks use dynamic memory allocation and can grow or shrink as needed.

Key Concept: The top of the stack is represented by the head of the linked list. All insertions and deletions occur at the beginning of the list.

Why Linked Implementation?

- Overcomes fixed size limitation of arrays
- Dynamic size - grows and shrinks as needed
- No stack overflow (until system memory exhausted)
- Efficient memory utilization for varying sizes

2. Node Structure Definition

Basic Node Structure:

```c
```

```c
struct node {
    int data;          // Data part
    struct node *next;  // Link to next node
};
```

Stack Pointers:

```c
struct node *front;  // Special header node
struct node *rear;   // Not used in stack (used in queue)
```

Initialization:

```c
// In main() or initialization function
front = (struct node *) malloc(sizeof(struct node));
front->next = NULL;
rear = NULL;
```

Visual Representation:

Empty Stack:

```
front → [header|NULL]
rear = NULL
```

Stack with elements (Top to Bottom):

```
front → [header|•] → [10|•] → [20|•] → [30|NULL]
                      ↑
                     top
```

3. Complete Structure and Global Declarations

```c
```

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct node {
    int data;
    struct node *next;
};

// Global pointers
struct node *front, *rear;

// Function prototypes
void initStack();
int isEmpty();
void push(int element);
int pop();
int peek();
void display();
```

4. Stack Initialization

Purpose: Set up empty stack with header node.

```c
void initStack() {
    front = (struct node *) malloc(sizeof(struct node));
    if (front == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    front->next = NULL;
    rear = NULL;
}
```

After Initialization:

```
Memory Layout:
+--------+------+
| data   | next |  ← front (header node)
+--------+------+
   ?       NULL

Stack is empty when: front->next == NULL
```

5. isEmpty() Operation

Purpose: Check if stack has no elements.

Algorithm:

```
Algorithm isEmpty()
{
   if (front->next == NULL)
      return 1;  // true - stack is empty
   else
      return 0;  // false - stack has elements
}
```

C Implementation:

```c
int isEmpty() {
   if (front->next == NULL)
      return 1;
   else
      return 0;
}
```

Time Complexity: O(1)
Space Complexity: O(1)

6. Push Operation (Enqueue at Top)

Purpose: Insert new element at the top of stack.

Algorithm:

```
Algorithm push(element)
{
   // Step 1: Allocate memory for new node
   new_node = (struct node *) malloc(sizeof(struct node));

   // Step 2: Initialize new node
   new_node->data = element;
   new_node->next = NULL;

   // Step 3: Check if stack is empty
   if (isEmpty())
   {
      // First element - both front and rear point to it
      front->next = new_node;
      rear = new_node;
   }
   else
   {
      // Insert at beginning (top of stack)
      new_node->next = front->next;
      front->next = new_node;
   }
}
```

C Implementation:

```
c
```

```
void push(int element) {
    struct node *new_node;

    // Allocate memory
    new_node = (struct node *) malloc(sizeof(struct node));

    if (new_node == NULL) {
        printf("Stack Overflow - Memory allocation failed\n");
        return;
    }

    // Initialize new node
    new_node->data = element;
    new_node->next = NULL;

    // Check if empty
    if (isEmpty()) {
        front->next = new_node;
        rear = new_node;
    }
    else {
        new_node->next = front->next;
        front->next = new_node;
    }

    printf("Pushed %d\n", element);
}
```

Visual Demonstration:

Step 1: Empty Stack

```
front → [H|NULL]
rear = NULL
```

Step 2: Push(10)

```
Before:
front → [H|NULL]
new_node → [10|NULL]

After:
front → [H|•] → [10|NULL]
          ↑
         rear
```

Step 3: Push(20)

```
Before:
front → [H|•] → [10|NULL]
new_node → [20|NULL]

After:
front → [H|•] → [20|•] → [10|NULL]
         ↑top    ↑rear
```

Step 4: Push(30)

```
After:
front → [H|•] → [30|•] → [20|•] → [10|NULL]
         ↑top           ↑rear
```

Time Complexity: O(1)

Space Complexity: O(1) per operation, O(n) total for n elements

7. Pop Operation (Delete from Top)

Purpose: Remove and return the top element from stack.

Algorithm:

```
Algorithm pop()
{
   // Step 1: Check if stack is empty
   if (isEmpty())
   {
      print "Stack Underflow";
      return -1;
   }

   // Step 2: Store top node reference
   temp = front->next;

   // Step 3: Get value to return
   value = temp->data;

   // Step 4: Update front pointer
   front->next = temp->next;

   // Step 5: Free the node
   temp->next = NULL;
   free(temp);

   // Step 6: Return the value
   return value;
}
```

C Implementation:

```c

```

```c
int pop() {
    struct node *temp;
    int value;

    // Check if empty
    if (isEmpty()) {
        printf("Stack Underflow - Stack is empty\n");
        return -1;
    }

    // Get reference to top node
    temp = front->next;
    value = temp->data;

    // Update front pointer
    front->next = temp->next;

    // Free memory
    temp->next = NULL;
    free(temp);

    printf("Popped %d\n", value);
    return value;
}
```

Visual Demonstration:

Before Pop:

```
front → [H|•] → [30|•] → [20|•] → [10|NULL]
                ↑top
temp will point here
```

After Pop (removing 30):

```
front → [H|•] → [20|•] → [10|NULL]
                ↑top

temp → [30|NULL] (freed)
```

Time Complexity: O(1)

Space Complexity: O(1)

8. Peek/Top Operation

Purpose: View top element without removing it.

```c
int peek() {
    if (isEmpty()) {
        printf("Stack is empty\n");
        return -1;
    }
    return front->next->data;
}
```

Time Complexity: O(1)

9. Display Operation

Purpose: Print all elements in stack from top to bottom.

```c
void display() {
    struct node *temp;

    if (isEmpty()) {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements (Top to Bottom): ");
    temp = front->next;

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

Time Complexity: O(n) where n is number of elements

10. Complete Working Program

```c
```

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct node {
    int data;
    struct node *next;
};

// Global pointers
struct node *front, *rear;

// Initialize stack
void initStack() {
    front = (struct node *) malloc(sizeof(struct node));
    front->next = NULL;
    rear = NULL;
}

// Check if empty
int isEmpty() {
    return (front->next == NULL);
}

// Push operation
void push(int element) {
    struct node *new_node = (struct node *) malloc(sizeof(struct node));

    if (new_node == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    new_node->data = element;
    new_node->next = NULL;

    if (isEmpty()) {
        front->next = new_node;
        rear = new_node;
    } else {
        new_node->next = front->next;
        front->next = new_node;
    }

    printf("Pushed: %d\n", element);
}
```

```c
// Pop operation
int pop() {
    if (isEmpty()) {
        printf("Stack Underflow\n");
        return -1;
    }

    struct node *temp = front->next;
    int value = temp->data;

    front->next = temp->next;
    free(temp);

    printf("Popped: %d\n", value);
    return value;
}

// Peek operation
int peek() {
    if (isEmpty()) {
        printf("Stack is empty\n");
        return -1;
    }
    return front->next->data;
}

// Display stack
void display() {
    if (isEmpty()) {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack (Top to Bottom): ");
    struct node *temp = front->next;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Main function
int main() {
    int choice, value;
```

```
    initStack();

    while (1) {
        printf("\n=== Linked Stack Operations ===\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                value = pop();
                if (value != -1)
                    printf("Popped value: %d\n", value);
                break;
            case 3:
                value = peek();
                if (value != -1)
                    printf("Top value: %d\n", value);
                break;
            case 4:
                display();
                break;
            case 5:
                printf("Exiting...\n");
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

11. Step-by-Step Example Execution

Sequence: Push(10), Push(20), Push(30), Pop(), Pop(), Push(40), Display()

```
Operation: initStack()
State: front → [H|NULL], rear = NULL
---

Operation: Push(10)
Before: front → [H|NULL]
After:  front → [H|•] → [10|NULL]
                ↑rear
---

Operation: Push(20)
Before: front → [H|•] → [10|NULL]
After:  front → [H|•] → [20|•] → [10|NULL]
                ↑top     ↑rear
---

Operation: Push(30)
After:  front → [H|•] → [30|•] → [20|•] → [10|NULL]
                ↑top             ↑rear
---

Operation: Pop()
Returned: 30
After:  front → [H|•] → [20|•] → [10|NULL]
                ↑top     ↑rear
---

Operation: Pop()
Returned: 20
After:  front → [H|•] → [10|NULL]
                ↑top/rear
---

Operation: Push(40)
After:  front → [H|•] → [40|•] → [10|NULL]
                ↑top     ↑rear
---

Operation: Display()
Output: Stack (Top to Bottom): 40 10
```

12. Advantages of Linked Stack

Compared to Array Implementation:

1. **Dynamic Size**:

- No fixed size limitation
- Grows and shrinks as needed
- No memory wastage

2. **No Stack Overflow** (in practical sense):
   - Only limited by system memory
   - Array stack has fixed MAX_SIZE
3. **Efficient Memory Usage**:
   - Allocates exactly what's needed
   - No pre-allocated unused space
4. **Easy Implementation**:
   - No need to track top index
   - Simple pointer manipulation
5. **No Size Declaration**:
   - Don't need to predict maximum size
   - Flexible for varying requirements

13. Disadvantages of Linked Stack

Compared to Array Implementation:

1. **Extra Memory**:
   - Each node needs pointer field (extra 4-8 bytes)
   - Header node overhead
2. **Slower Access**:
   - Dynamic memory allocation overhead
   - Pointer dereferencing
   - Cache performance issues
3. **No Random Access**:
   - Cannot access arbitrary elements
   - Must traverse from top
4. **Memory Fragmentation**:
   - Nodes scattered in memory
   - May cause fragmentation
5. **Complex Implementation**:
   - More code than array version
   - Pointer management complexity

14. Comparison: Array vs Linked Stack

Feature	Array Stack	Linked Stack
Size	Fixed	Dynamic
Memory	Contiguous	Scattered
Overflow	Possible	Rare
Space per element	sizeof(element)	sizeof(element) + sizeof(pointer)
Access speed	Faster (O(1))	Slightly slower
Implementation	Simpler	More complex
Memory waste	If underutilized	Minimal
Cache performance	Better	Worse
Suitable for	Known max size	Unknown/varying size

15. Time and Space Complexity Summary

Linked Stack Operations:

Operation	Time Complexity	Space Complexity
Push	O(1)	O(1)
Pop	O(1)	O(1)
Peek/Top	O(1)	O(1)
isEmpty	O(1)	O(1)
Display	O(n)	O(1)

Overall Space: O(n) where n = number of elements

16. Common Errors and Solutions

Error 1: Memory Leak

```
c
```

```c
// Wrong - doesn't free memory
int pop() {
    temp = front->next;
    front->next = temp->next;
    return temp->data;  // Memory leak!
}

// Correct
int pop() {
    temp = front->next;
    value = temp->data;
    front->next = temp->next;
    free(temp);  // Free memory
    return value;
}
```

Error 2: Null Pointer Dereference

```c
// Wrong - doesn't check isEmpty
int peek() {
    return front->next->data;  // Crash if empty!
}

// Correct
int peek() {
    if (isEmpty()) return -1;
    return front->next->data;
}
```

Error 3: Not Initializing new_node->next

```c
// Wrong
new_node->data = element;
// Forgot: new_node->next = NULL;

// Correct
new_node->data = element;
new_node->next = NULL;
```

17. Applications

Linked Stacks are Preferred When:

- Maximum size unknown at compile time
- Stack size varies significantly during execution
- Memory efficiency crucial for small stacks
- Need to avoid stack overflow errors
- Implementing multiple stacks dynamically

Examples:

- Expression evaluation with unknown complexity
- Recursive function call management
- Undo mechanisms in applications
- Backtracking algorithms
- Browser history with dynamic size

Q6: Explain Queue as an Abstract Data Type (ADT) with its operations, sequential organization using arrays, circular queue concept, advantages, and complete implementation with examples.

Answer:

1. Queue as an Abstract Data Type

A **queue** is an ordered linear data structure that follows the **First In First Out (FIFO)** principle. Elements are inserted at one end (rear) and deleted from the other end (front).

Definition: A queue is a collection of elements where:

- Insertions (Enqueue) occur at the REAR end
- Deletions (